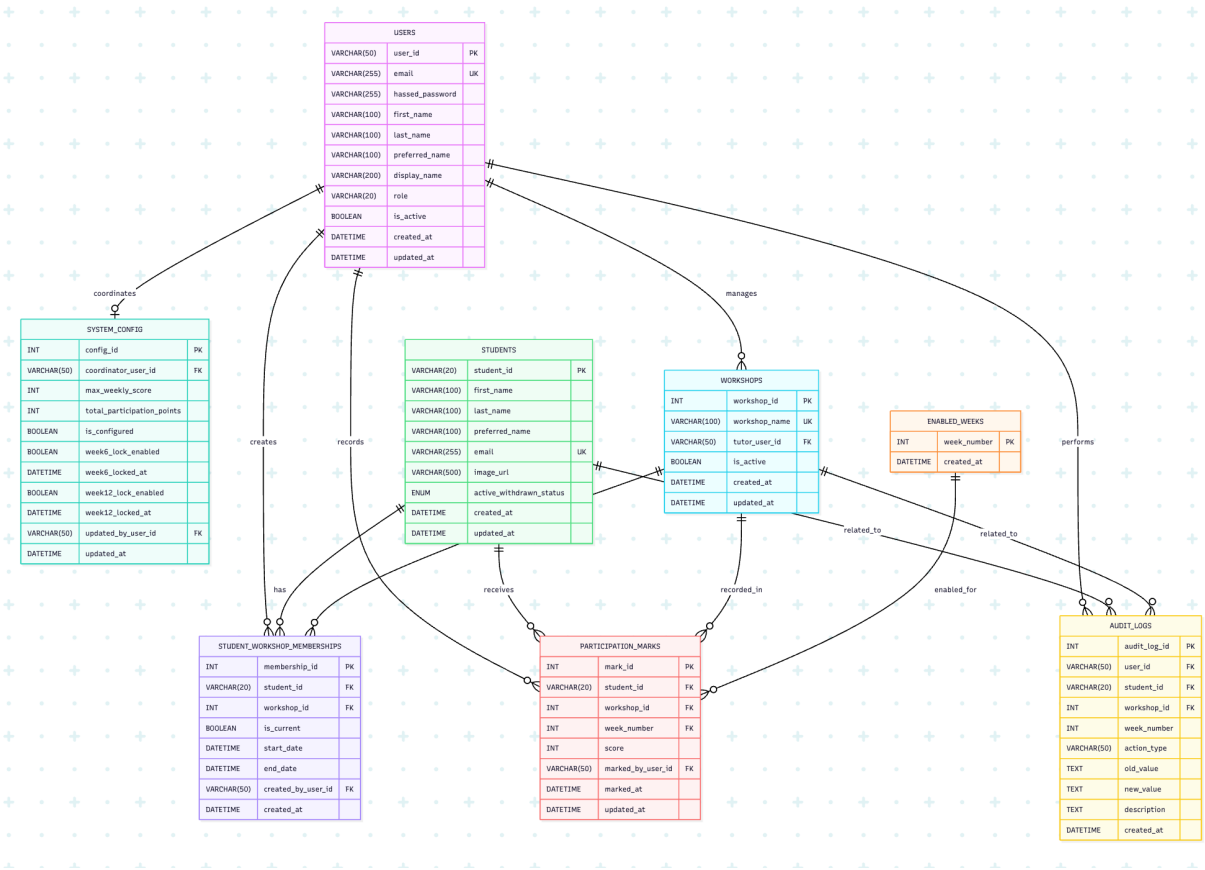
ER Diagram V3 (using Mermaid):



V1 01/04/2026:

1. Users are limited to two valid roles: coordinator and tutor.
2. Each tutor may manage one or more workshops, while each workshop has at most one current tutor.
3. Students may be added, moved between workshops, or withdrawn, and only one current workshop membership should exist per student at any time.
4. Student status is limited to active or withdrawn.
5. Participation marking is recorded per student, per enabled week, and each student may have only one final participation mark for a given week.
6. Participation scores must be restricted to values from 0 to 3.
7. Only weeks listed in ENABLED_WEEKS should be used for marking, with this rule enforced by application logic.
8. Audit history is required, and AUDIT_LOGS.student_id and AUDIT_LOGS.workshop_id are optional depending on the action type.

V2 12/04/2026:

1. login now needs a **hashed_password**
2. there are still only two roles: UC and tutor
3. A user with role UC may be allowed by application logic to perform tutor-related actions, while a user with role tutor cannot perform UC-only actions.
4. A semester-based design was explored in this version, but it was later removed in V3 when the schema was simplified to support the current semester only.

V3 19/04/2026 (all the update in the following schema has been **Bold**):

1. status ENUM('active', 'withdrawn') NOT NULL DEFAULT 'active'
2. In enables_weeks table, Remove enabled_week_id and is_enabled. Week_number is used directly as the primary key for the current-semester design.
3. The system only manages participation data for the current semester. Historical semester data is not retained in this Participation Mark system after final export to the official UWA system at the end of a semester.

Schema:

1. users

Purpose: stores staff accounts only, including the single coordinator and all tutors.

Fields:

- user_id VARCHAR(50) PK
- email VARCHAR(255) UNIQUE NOT NULL
- **hashed_password VARCHAR(255) NOT NULL**
- first_name VARCHAR(100) NOT NULL
- last_name VARCHAR(100) NOT NULL
- preferred_name VARCHAR(100) NULL
- display_name VARCHAR(200) NOT NULL
- role VARCHAR(20) NOT NULL
- is_active BOOLEAN NOT NULL DEFAULT TRUE
- created_at DATETIME NOT NULL
- updated_at DATETIME NOT NULL

Constraints

- role should only allow 'UC' or 'tutor'.

A user with role UC may be allowed by application logic to perform tutor-related actions, while a user with role tutor cannot perform UC-only actions.

Why this design
This table is kept simple because staff roles are fixed and clear in the project. A single role field is still enough. **In this update, hashed_password is added because login now requires a password. Using a hash rather than storing a plain password is safer and more realistic.**

2. system_config

Purpose: stores the global configuration for the current semester's participation marking system of this unit. The table is expected to contain a single row for the current semester configuration.

Fields:

- **config_id INT PK**
- coordinator_user_id VARCHAR(50) FK -> users.user_id

- max_weekly_score INT NOT NULL DEFAULT 3
- total_participation_points INT NOT NULL DEFAULT 0
- is_configured BOOLEAN NOT NULL DEFAULT FALSE
- week6_lock_enabled BOOLEAN NOT NULL DEFAULT FALSE
- week6_locked_at DATETIME NULL
- week12_lock_enabled BOOLEAN NOT NULL DEFAULT FALSE
- week12_locked_at DATETIME NULL
- updated_by_user_id VARCHAR(50) FK -> users.user_id NULL
- updated_at DATETIME NOT NULL

Why this design
The system only manages the current semester, so one central configuration table is enough. This table stores the main marking rules and lock settings used by the coordinator in the Settings screen

3. enabled_weeks

Purpose: stores which teaching weeks are enabled for participation marking.

Fields:

- **week_number INT PK**
- created_at DATETIME NOT NULL

Why this design

Using week_number directly as the primary key is sufficient because the system only operates for the current semester. This keeps the schema simple and avoids introducing an unnecessary artificial identifier. It also allows participation_marks to reference enabled weeks directly.

4. workshops

Purpose: stores all workshop groups and their currently assigned tutor.

Fields:

- workshop_id INT PK
- workshop_name VARCHAR(100) NOT NULL UNIQUE
- tutor_user_id VARCHAR(50) FK -> users.user_id NULL
- is_active BOOLEAN NOT NULL DEFAULT TRUE
- created_at DATETIME NOT NULL
- updated_at DATETIME NOT NULL

Why this design
Each workshop has one current tutor, while one tutor may own multiple workshops. This one-to-many structure matches our actual business rule and supports workshop creation, tutor assignment, and workshop detail views.

5. students

Purpose: stores each student's core profile information.

Fields:

- student_id VARCHAR(20) PK
- first_name VARCHAR(100) NOT NULL
- last_name VARCHAR(100) NOT NULL
- preferred_name VARCHAR(100) NULL
- email VARCHAR(255) UNIQUE NOT NULL
- image_url VARCHAR(500) NULL
- status ENUM('active', 'withdrawn') NOT NULL DEFAULT 'active'
- created_at DATETIME NOT NULL
- updated_at DATETIME NOT NULL

Why this design
This table follows the agreed CSV template and keeps student identity independent from workshop placement. That separation is important because students may later move between workshops without changing their core record.

6. student_workshop_memberships

Purpose: stores current and past workshop allocation for each student.

Fields:

- membership_id INT PK
- student_id VARCHAR(20) FK -> students.student_id
- workshop_id INT FK -> workshops.workshop_id
- is_current BOOLEAN NOT NULL DEFAULT TRUE
- start_date DATETIME NOT NULL
- end_date DATETIME NULL
- created_by_user_id VARCHAR(50) FK -> users.user_id NULL
- created_at DATETIME NOT NULL

Why this design

This table is included because our system must support Move to Workshop properly. It preserves allocation history and avoids the weakness of storing only one workshop_id directly in the student table.

7. participation_marks

Purpose: stores the final participation score for each student in each enabled week.

Fields:

- mark_id INT PK
- student_id VARCHAR(20) FK -> students.student_id
- workshop_id INT FK -> workshops.workshop_id
- week_number INT NOT NULL
- score INT NOT NULL
- marked_by_user_id VARCHAR(50) FK -> users.user_id
- marked_at DATETIME NOT NULL
- updated_at DATETIME NOT NULL

Constraints

- UNIQUE (student_id, week_number)
- week_number INT NOT NULL FK -> enabled_weeks.week_number
- score should be restricted to values between 0 and 3

Why this design

Marking is done per student, per enabled week, and each student should only have one final mark for a given week. Linking week_number directly to enabled_weeks ensures that only enabled weeks can be used for marking. Keeping workshop_id here also supports analytics, tutor accountability, and audit tracing.

8. audit_logs

Purpose: stores important system actions for traceability and review.

Fields:

- audit_log_id INT PK
- user_id VARCHAR(50) FK -> users.user_id
- student_id VARCHAR(20) FK -> students.student_id NULL
- workshop_id INT FK -> workshops.workshop_id NULL
- week_number INT NULL

- action_type VARCHAR(50) NOT NULL
- old_value TEXT NULL
- new_value TEXT NULL
- description TEXT NOT NULL
- created_at DATETIME NOT NULL

Example action_type values (just some examples)

- config_updated
- workshop_created
- tutor_changed
- student_added
- student_moved
- student_withdrawn
- mark_created
- mark_updated

Why this design
Our Figma design shows that audit history matters. This table keeps a lightweight but useful record of major changes without making the system unnecessarily complex.

Appendix:

I. Intermediate table

student_workshop_memberships

It connects students and workshops and records which workshop a student belongs to over time.

Why not store workshop_id directly in students:
Because the system supports Move to Workshop. If workshop membership were stored only in the students table, moving a student would overwrite the old assignment and lose history.

This table allows: tracking the student's current workshop; preserving past workshop history; supporting clean student transfer logic; and keeping the student identity separate from workshop allocation.

II. One-to-many relationships

1. users → workshops

One tutor can manage many workshops.
Each workshop has one current tutor, but a tutor may be assigned to multiple workshops.

2. users → participation_marks

One tutor can create many participation marks.
Each mark stores who recorded it, so one user may be linked to many marking records.

3. users → audit_logs

One user can generate many audit log entries.
This supports traceability for actions such as configuration updates, student transfers, and mark changes.

4. students → student_workshop_memberships

One student can have many workshop membership records over time.
Normally only one record is current, but older records are kept to preserve transfer history.

5. workshops → student_workshop_memberships

One workshop can contain many students.
This table represents all students currently or previously assigned to that workshop.

6. students → participation_marks

One student can have many participation marks.
A student may receive one mark for each enabled week.

7. workshops → participation_marks

One workshop can have many participation marks.
All student marks recorded within that workshop are linked back to it.

8. students → audit_logs

One student can appear in many audit log records.
For example, adding, moving, withdrawing, or updating marks for a student may all create log entries.

9. workshops → audit_logs

One workshop can appear in many audit log records.
This helps record workshop-related actions such as tutor changes or student transfers.